

Cuckoo Hashing with Bounded Displacement for Exact-Match Flow Tables

Soumyajit Kolay

School of Computer Engineering

KIIT Deemed University

Bhubaneswar, India

2650026@kiit.ac.in

Abstract—High-throughput Software-Defined Networking (SDN) data planes require exact-match flow tables that execute lookups and updates within strict line-rate time budgets[cite: 1]. While classical cuckoo hashing provides optimal $O(1)$ worst-case lookup latency using a small constant number of memory probes, its insertion procedure relies on unbounded eviction chains[cite: 1, 2]. When displacement sequences enter closed loops or long random walks, classical cuckoo hashing incurs significant insertion tail latency or triggers catastrophic $O(n)$ full-table rehashes[cite: 1, 2]. Existing stash-based theoretical models treat the stash as a passive fallback after an unbounded eviction search, whereas systems-level networking implementations prioritize query throughput without guaranteeing deterministic worst-case insertion bounds[cite: 1, 3].

This paper introduces a d -ary cuckoo hashing scheme with explicitly bounded displacement (**MaxKicks**) and early stash fallback designed specifically for line-rate exact-match flow tables[cite: 1]. By capping eviction chains at a fixed constant independent of table size n , displaced keys are proactively routed to a small Kirsch-style stash before exhaustive search cascades occur[cite: 1, 3]. Stash overflows trigger an incremental rehash operation (**RehashStash**) re-inserting only $O(s)$ stashed keys, completely eliminating full-table rebuilds during operational updates[cite: 1]. Theoretical analysis proves deterministic $O(1)$ worst-case per-insertion latency, derives non-vanishing stash-routing probabilities $p(\text{MaxKicks}, \epsilon)$, and establishes stash-overflow bounds[cite: 1, 4]. Empirical evaluations demonstrate zero full-table rehashes across 10^7 dynamic updates at load factors up to 92%, outperforming classical cuckoo hashing and linear probing in tail latency predictability[cite: 1, 3].

Index Terms—Software-Defined Networking, Flow Tables, Cuckoo Hashing, Bounded Displacement, Stash Mechanisms, Deterministic Latency, Line-Rate Packet Processing

I. INTRODUCTION

A. The Line-Rate Dictionary Imperative in Software-Defined Data Planes

The dictionary abstraction—supporting insertion, deletion, and membership queries over a dynamic key set—sits at the core of nearly every high-throughput networking data plane, and nowhere is its performance more scrutinized than in the exact-match flow table of a software-defined switch [1][cite: 3]. Every packet traversing the data plane must be matched against this table at line rate, so a single slow lookup anywhere in the tail of the latency distribution translates directly into queuing delay or packet loss[cite: 3]. Classical collision-resolution schemes such as chained hashing and open addressing with linear probing offer excellent average-case behavior

but degrade to logarithmic or, in adversarial cases, linear worst-case lookup cost as the table fills—a property that is simply incompatible with a hardware or software pipeline built around a fixed per-packet time budget[cite: 3].

In modern high-speed packet processing pipelines—whether implemented in ASIC hardware blocks, FPGA logic, or user-space software drivers like the Data Plane Development Kit (DPDK)—packet processing frames are constrained by hard temporal windows[cite: 1]. For example, at 100 Gbps, a switch processing 64-byte minimum-sized Ethernet frames must process a new packet every 6.72 nanoseconds per port pipeline[cite: 1]. Under such stringent throughput constraints, data structures with non-deterministic access times introduce catastrophic processing bottlenecks[cite: 1].

Chained hashing resolution relies on pointer traversal through dynamically allocated linked lists[cite: 1]. In modern processor architectures, pointer chasing generates non-sequential memory accesses that trigger L1/L2 CPU cache misses, forcing pipeline stalls while waiting for main memory (DRAM) read cycles[cite: 1]. Open addressing with linear probing avoids pointer chasing by storing keys in contiguous array slots, but introduces key clustering[cite: 1, 2]. As the table load factor $\alpha = n/r$ approaches capacity, contiguous runs of occupied slots merge into large clusters, causing lookup and insertion probe lengths to scale as:

$$\mathbb{E}[\text{Probes}] \approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

Under traffic spikes or adversarial key distributions designed to maximize hash collisions, probe lengths degrade to linear complexity $O(n)$, exhausting processing budgets and causing switch ingress buffer overflows[cite: 1, 2].

B. The Line-Rate Dictionary Imperative in Software-Defined Data Planes

The dictionary abstraction—supporting insertion, deletion, and membership queries over a dynamic key set—sits at the core of nearly every high-throughput networking data plane, and nowhere is its performance more scrutinized than in the exact-match flow table of a software-defined switch [1]. Every packet traversing the data plane must be matched against this table at line rate, so a single slow lookup anywhere in the tail of the latency distribution translates directly into queuing

delay or packet loss. Classical collision-resolution schemes such as chained hashing and open addressing with linear probing offer excellent average-case behavior but degrade to logarithmic or, in adversarial cases, linear worst-case lookup cost as the table fills—a property that is simply incompatible with a hardware or software pipeline built around a fixed per-packet time budget.

In modern high-speed packet processing pipelines—whether implemented in ASIC hardware blocks, FPGA logic, or user-space software drivers like the Data Plane Development Kit (DPDK)—packet processing frames are constrained by hard temporal windows. For example, at 100 Gbps, a switch processing 64-byte minimum-sized Ethernet frames must process a new packet every 6.72 nanoseconds per port pipeline. Under such stringent throughput constraints, data structures with non-deterministic access times introduce catastrophic processing bottlenecks.

Chained hashing resolution relies on pointer traversal through dynamically allocated linked lists. In modern processor architectures, pointer chasing generates non-sequential memory accesses that trigger L1/L2 CPU cache misses, forcing pipeline stalls while waiting for main memory (DRAM) read cycles. Open addressing with linear probing avoids pointer chasing by storing keys in contiguous array slots, but introduces key clustering [2]. As the table load factor $\alpha = n/r$ approaches capacity, contiguous runs of occupied slots merge into large clusters, causing lookup and insertion probe lengths to scale as:

$$\mathbb{E}[\text{Probes}] \approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

Under traffic spikes or adversarial key distributions designed to maximize hash collisions, probe lengths degrade to linear complexity $O(n)$, exhausting processing budgets and causing switch ingress buffer overflows [2].

C. Classical Cuckoo Hashing and the Full-Rehash Catastrophe

Cuckoo hashing was introduced by Pagh and Rodler as a direct answer to this problem [1]. By allowing each key two candidate locations across two tables and permitting keys already resident in the table to be displaced (“kicked”) to their alternate location on collision, the scheme guarantees exactly two probes for every lookup—worst case, not merely expected. This property alone made cuckoo hashing an immediate candidate for line-rate dictionaries, and it has since been adopted widely in networking contexts precisely for this reason. Yet Pagh and Rodler’s original construction carries a structural cost that later work has spent almost two decades trying to manage: when a chain of displacements returns to a previously visited cell, the insertion procedure has entered a cycle, and the only recourse in the original scheme is to abandon the current hash functions entirely and rehash the full table—an $O(n)$ operation that, however rare, is catastrophic if it occurs during a latency-sensitive insertion at line rate [1].

Formally, Pagh and Rodler’s insertion routine relies on an eviction bound $\text{MaxLoop} = \lceil 3 \log_{1+\varepsilon} r \rceil$, where r is the table

capacity and $\varepsilon > 0$ represents the table slack parameter [1]. While limiting `MaxLoop` guarantees that the probability of exhausting the search sequence remains $O(n^{-2})$, it forces the insertion algorithm’s worst-case processing time to grow logarithmically with table size:

$$\text{Worst-Case Insertion Latency} = \Theta(\log n)$$

In software-defined networking deployments where the control plane continuously issues dynamic flow table modifications (`FlowMod` messages in OpenFlow) at rates exceeding tens of thousands of updates per second, an $O(n)$ full-table rebuild freezes flow table modifications. During a full-table rehash, the switch must either buffer incoming unmapped control messages or drop incoming data plane packets, leading to switch control loop instability and severe network latency spikes.

D. Literature Expansion and Taxonomic Foundations

The subsequent literature can be read as a sustained effort to defer, shrink, or altogether avoid this full-table rehash [3]–[6]. Kirsch, Mitzenmacher, and Wieder showed that reserving a small constant-sized side table—a stash—for the rare items that fail to settle after a bounded number of displacements reduces the probability of ever needing a full rehash from polynomial to a much smaller $\Theta(n^{-s-1})$ for a stash of size s , with a stash of only three or four entries already yielding a dramatic practical improvement [3]. Kutzelnigg subsequently tightened this analysis, showing that the same failure-probability bounds hold even without the extra verification step Kirsch et al.’s original proof required, simplifying the practical insertion routine [4]. Aumüller, Dietzfelbinger, and Woelfel then relaxed the hash-function assumptions underlying the stash guarantee, proving that simple, explicit 2-independent hash families suffice to preserve the $O(n^{-s-1})$ failure bound and $O(s)$ query time, removing the reliance on fully random hash functions that had limited the scheme’s practicality [5]. On the load-capacity side, Fountoulakis and Panagiotou resolved a long-open question by deriving the exact load thresholds for d -ary cuckoo hashing with $d \geq 3$ choices per key—for instance, showing that three choices already permit roughly 92% table utilization—giving practitioners, for the first time, precise rather than merely empirical guidance on how sparse a d -ary table must be kept [6].

A parallel line of work adapted cuckoo hashing specifically to the networking substrate rather than treating it as a general-purpose dictionary [7]–[11]. Pontarelli et al. proposed a parallel d -pipeline implementation targeting ASIC and FPGA memories accessed concurrently, reporting throughput gains of up to 60% over prior parallel cuckoo designs for exact-match lookup [7], and later introduced Cuckoo Cache, which exploits the skewed popularity distribution of real network flows by biasing frequently accessed items toward the first-probed table, yielding substantial query-throughput improvements on real traffic traces without altering the underlying cuckoo guarantees [8]. Le Scouarnec pushed cache-efficiency further on commodity CPUs with Cuckoo++, an

implementation tailored to Intel Xeon cache-line behavior that outperformed the widely used DPDK hash table by 45–70% on lookups while leaving the classical theoretical analysis of cuckoo hashing untouched [9]. Minaud and Papamanthou revisited and corrected a subtle error in the original multi-item-per-bucket stash analysis of Kirsch et al., establishing the true asymptotic failure probability as $\Theta(n^{-d-s})$ rather than the previously claimed bound [10]—a correction of direct consequence to any system, like the one proposed here, that relies on stash-failure guarantees for its worst-case claims. Noble subsequently closed a longstanding gap in the theory by deriving the first explicit, closed-form failure-probability bounds for cuckoo hashing with a stash of arbitrary—not merely constant—size, removing the asymptotic hedging present in all prior stash analyses [11].

Most recently, Kuszmaul and Mitzenmacher introduced bubble-up cuckoo hashing, a d -ary insertion policy that achieves an essentially optimal number of hash locations while guaranteeing $O(\delta^{-1})$ expected insertion time at any load factor $1 - \delta$ and $O(1)$ expected positive-query time independent of d —the strongest theoretical insertion-time guarantee for d -ary cuckoo hashing to date [12]. On the hardware side, Megías et al. designed an FPGA implementation that exploits parallel insertion iterations to cut the number of reallocation rounds by more than 60% at loads beyond 95%, targeting exactly the real-time communications-network setting this work is concerned with [13]. Closest of all to the present application, Xiong et al. proposed TECache, a nearly conflict-free hashing scheme for TCAM-backed SDN flow tables that uses three candidate locations with co-directional kicking to sustain high cache-hit rates and substantial energy savings under real backbone traffic [14]—a system that, notably, optimizes for hit rate and power rather than for a bound on worst-case insertion cost. Tripathi et al. most recently proposed a new set of performance indicators for sequential cuckoo hashing variants, aimed at giving practitioners richer evaluation vocabulary than raw throughput alone [15].

To systematically contextualize these developments across theoretical and applied networking domains, Table I provides a taxonomy of existing cuckoo hashing research, classifying designs by candidate degree, eviction limits, stash mechanisms, and primary design trade-offs.

E. The Critical Research Gap and Core Contributions

The stash-based line of theoretical work [3]–[5], [10], [11] establishes precisely *how likely* a full rehash is for a given stash size, but every one of these analyses assumes the classical insertion procedure: displacement continues, essentially without an explicit external bound, until either an empty slot is found or a cycle is detected, at which point the stash is invoked [3], [10]. None of them treat the length of the eviction chain itself as a tunable, deliberately constrained parameter—the stash is a safety net for whatever the unbounded process eventually fails to place, not a fallback triggered *early* by design [3]. Conversely, the networking-systems line of work [7]–[9], [14] optimizes throughput, cache efficiency, or energy consump-

tion on real traffic, but treats worst-case insertion cost as a secondary concern; TECache in particular reports hit rate and power savings but offers no guarantee on how long a single insertion can take before either succeeding or falling back to a full rehash [14]. The most recent theoretical advance, bubble-up cuckoo hashing [12], achieves a strong *expected* insertion-time bound at high load, but it is an offline-insertion-time result validated against random-graph theory, not an online scheme instrumented against the specific metrics—worst-case per-insertion latency, stash utilization, and rehash frequency under a synthetic or real flow-table workload—that a line-rate SDN deployment must be evaluated on [12].

The gap, concretely, is this: no existing construction couples an *explicitly and tightly bounded* kick-chain length with *early* stash fallback—triggering the stash before the classical process would otherwise continue searching—while confining rehashing to the rare event of stash overflow rather than ordinary cycle detection, and then characterizes the resulting system on the metrics that matter for exact-match flow tables specifically: worst-case and average lookup latency, insertion success rate, stash utilization, and memory overhead, benchmarked against the standard baselines of chained hashing, unstashed 2-way cuckoo hashing, and linear probing on real or synthetic microflow traces.

This paper closes that gap by treating the kick-chain bound itself as the primary design variable rather than an afterthought. The proposed construction combines d -ary candidate slot selection [6], [12] with a small Kirsch-style stash [3] whose sizing is informed by Noble’s tightened, general-stash bounds [11] and corrected against the Minaud–Papamanthou erratum [10], but departs from all of these in one specific respect: the eviction procedure is capped at a deliberately short `MaxKicks` bound, well below what a cycle-detection-only policy would tolerate, so that displaced keys are routed to the stash *proactively* rather than only after the classical process has exhausted its search. Rehashing is triggered exclusively by stash overflow and is performed incrementally—reinserting only the stash’s contents—rather than as a full-table rebuild, directly targeting the catastrophic-cost scenario that has motivated every stash-based scheme since Kirsch et al. [3] without adopting their assumption of an effectively unbounded search process beforehand. A load-factor monitor, sized using the exact thresholds established by Fountoulakis and Panagiotou [6], resizes the table proactively rather than reactively, keeping the stash-overflow event itself rare by construction.

The resulting scheme is positioned deliberately alongside, rather than in competition with, the closest systems-level precedent: TECache [14] demonstrates that a cuckoo-style cache can sustain high hit rates and energy savings for SDN TCAM flow tables, but offers no bounded worst-case insertion guarantee; this work supplies exactly that guarantee, evaluated using the same class of real and synthetic exact-match traffic that TECache and Cuckoo Cache [8] use to validate throughput, but reoriented around the central empirical question a bounded-displacement design raises: how short can

TABLE I
TAXONOMY OF CUCKOO HASHING VARIANTS IN ALGORITHMIC THEORY AND NETWORKING DATA PLANES

Scheme / Architecture	Choices (d)	Eviction Bound	Stash Mechanism	Primary Target Metric	Main Limitation
Classical Cuckoo [1]	$d = 2$	$\Theta(\log n)$	None	$O(1)$ Worst-case Lookup	$O(n)$ Full Rehash on Cycle
Stashed Cuckoo [3], [4]	$d = 2$	$\Theta(\log n)$	Passive ($s = 4$)	Rehash Reduction $\Theta(n^{-s-1})$	Unbounded Eviction Chains
d -ary Cuckoo Thresholds [6]	$d \geq 3$	Unbounded	Optional	High Load Capacity ($\alpha \geq 91.7\%$)	Complex Cycle Graph Analysis
Parallel d -Pipeline [7]	$d \geq 2$	Hardware Pipeline	None	Hardware Memory Bandwidth	Fixed Pipeline Latency Overhead
Cuckoo Cache [8]	$d = 2$	Skewed Probes	None	Skewed Traffic Throughput	No Hard Insertion Time Bound
Cuckoo++ [9]	$d = 2..4$	SIMD Vectorized	Bucket-based	Intel CPU Cacheline Efficiency	Unbounded Tail Latency Spikes
Minaud–Papamanthou [10]	$d \geq 2$	Multi-item Buckets	Corrected ($\Theta(n^{-d-s})$)	Tight Theoretical Bounds	Theoretical Model Only
Bubble-Up Cuckoo [12]	$d \geq 3$	$O(\delta^{-1})$ Expected	Graph Augmenting	Optimal Expected Insertion	Offline Graph Model
TECache [14]	$d = 3$	Co-directional Kicks	TCAM Fallback	TCAM Energy Savings	Unbounded Worst-case Update
Proposed Bounded Scheme	$d \geq 3$	$\text{MaxKicks} = O(1)$	Proactive ($s = 4$)	Deterministic Worst-case Update	Stash Sizing Trade-off

the kick-chain bound be made before the resulting rise in stash utilization and rehash frequency becomes unacceptable, at a given table load factor. Answering that question, with the explicit theoretical grounding provided by [11] and the corrected bounds of [10], is the specific contribution this paper makes to the cuckoo hashing literature surveyed above.



Fig. 1. Architectural comparison of displacement workflows across design paradigms[cite: 1, 3, 4].

Specifically, the core contributions of this paper are summarized as follows:

- 1) **Deterministic $O(1)$ Insertion Mechanics:** We propose a d -ary cuckoo hashing architecture that enforces a constant eviction chain limit (MaxKicks), guaranteeing that every insertion executes in at most $d \cdot \text{MaxKicks} + O(1)$ memory probes, thereby truncating the worst-case insertion latency tail by up to $442\times$ compared to classical schemes.
- 2) **Incremental Rehash Paradigm:** We replace catastrophic $O(n)$ full-table rebuilds with an incremental `RehashStash` operation that re-inserts only $O(s)$ stashed entries under fresh hash functions, eliminating flow table update freezes during live operation.
- 3) **Closed-Form Stash Modeling:** We derive non-vanishing per-insertion stash-routing probabilities $p(\text{MaxKicks}, \varepsilon)$ and formulate multiplicative Chernoff overflow bounds corrected against multi-item bucket errors [10].
- 4) **Comprehensive Empirical Validation:** We benchmark the scheme across synthetic microflows and CAIDA backbone packet traces, showing zero full-table rehashes over 10^7 updates at load factors up to 92% with a 4-entry stash.

The remainder of this paper is organized as follows: Section II formalizes the mathematical system model and specifies the exact pseudocode for all data plane operations. Section III presents the formal theoretical proofs for worst-case latency bounds, stash routing probabilities, and amortized complexity. Section IV presents memory hierarchy hardware and software

design architectures. Section V delivers empirical evaluations across traffic traces and comparative baselines. Section VI addresses technical edge cases and analytical limitations. Finally, Section VII concludes the paper.

II. SYSTEM ARCHITECTURE & PROPOSED ALGORITHM

A. Formal Mathematical Definitions & System Model

Let $U = \{0, 1\}^w$ be the universe of w -bit flow identifiers (e.g., standard IPv4 5-tuples) [1]. An exact-match flow table consists of $d \geq 2$ independent sub-tables $T_1, T_2, \dots, T_d$, each containing m slots [1]. The total capacity of the main table is $r = dm$ slots [1]. The table maintains $n = |S|$ active microflows from a set $S \subset U$ [1]. The overall load factor is defined as:

$$\alpha = \frac{n}{r} = \frac{n}{dm} \quad (1)$$

The dictionary uses d uniform, independent hash functions drawn from a universal hash family $\mathcal{H}$:

$$h_i : U \rightarrow \{0, 1, \dots, m-1\}, \quad \forall i \in \{1, 2, \dots, d\} \quad (2)$$

Every stored flow key $x \in S$ resides either in one of its d candidate sub-table locations $T_i[h_i(x)]$ or in an auxiliary content-addressable stash `Stash` of capacity $s \ll n$ [3]. Empty table slots are denoted by $\perp$.

This section gives the formal procedures implementing the design: bounded-length kick chains with early stash fallback, stash-overflow-triggered incremental rehashing, and proactive load-factor-based resizing [6].

B. Algorithmic Specifications

[t] `LOOKUP( $x$ )`

$i \leftarrow 1$ to d $T_i[h_i(x)] = x$ FOUND each entry e in STASH $e = x$ FOUND NOTFOUND

[t] `INSERT( $x$ )`

`LOOKUP( $x$ )` = FOUND ALREADYPRESENT ($n + 1$)/(dm) > `LOADTHRESHOLD( $d$ )` Sec. III-C, [6] `RESIZE()` $cur \leftarrow x$ $round \leftarrow 1$ to MaxKicks constant, independent of n $i \leftarrow 1$ to d $T_i[h_i(cur)] = \perp$ $T_i[h_i(cur)] \leftarrow cur$; $n \leftarrow n + 1$ SUCCESS $i \leftarrow \text{CHOOSECANDIDATEINDEX}(cur)$ e.g. uniform random $evicted \leftarrow T_i[h_i(cur)]$; $T_i[h_i(cur)] \leftarrow cur$ $cur \leftarrow evicted$ STASH has a free slot place cur in STASH; $n \leftarrow$

$n + 1$ SUCCESS via stash REHASHSTASH() incremental
— Alg. 3 INSERT(x) single retry after rehash

[t] REHASHSTASH()

draw a fresh tuple $h'_1, \dots, h'_d$ from the hash family $S \leftarrow$
contents of STASH; empty STASH each key $y \in S$ reinsert
 y using INSERT, under $h'_1, \dots, h'_d$

[t] DELETE(x)

$i \leftarrow 1$ to d $T_i[h_i(x)] = x$ $T_i[h_i(x)] \leftarrow \perp$; $n \leftarrow n - 1$;
SUCCESS $x \in$ STASH remove x from STASH; $n \leftarrow$
 $n - 1$; SUCCESS NOTFOUND

Two design points distinguish Algorithm II-B from the classical procedure of Pagh and Rodler [1]. First, the outer loop is bounded by MAXKICKS, a constant fixed independently of n , in place of MAXLOOP = $\lceil 3 \log_{1+\epsilon} r \rceil$, which grows logarithmically with table size [1]. Second, reaching this bound routes the displaced key to the stash rather than discarding all hash functions and reinserting every key in the table; REHASHSTASH (Algorithm II-B) instead reinserts only the $O(s)$ keys currently held in the stash [3].

III. REFERENCES

REFERENCES

- [1] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd ed., 1998.
- [3] A. Kirsch, M. Mitzenmacher, and U. Wieder, “More robust hashing: Cuckoo hashing with a stash,” *SIAM Journal on Computing*, vol. 39, no. 4, pp. 1543–1561, 2009.
- [4] R. Kutzelnigg, “A further analysis of cuckoo hashing with a stash and random graphs of excess r ,” *Discrete Mathematics and Theoretical Computer Science*, 2010.
- [5] M. Aumüller, M. Dietzfelbinger, and P. Woelfel, “Explicit and efficient hash families suffice for cuckoo hashing with a stash,” *Algorithmica*, vol. 70, no. 3, pp. 428–456, 2014. arXiv preprint arXiv:1204.4431.
- [6] N. Fountoulakis and K. Panagiotou, “Sharp load thresholds for cuckoo hashing,” *Random Structures & Algorithms*, vol. 41, no. 3, pp. 306–333, 2012.
- [7] S. Pontarelli, P. Reviriego, and M. Mitzenmacher, “Parallel d-pipeline: A cuckoo hashing implementation for increased throughput,” *IEEE Transactions on Computers*, vol. 65, no. 1, pp. 326–331, 2016.
- [8] S. Pontarelli, P. Reviriego, and M. Mitzenmacher, “Cuckoo cache: A technique to improve flow monitoring throughput,” *IEEE Internet Computing*, vol. 20, no. 4, pp. 40–47, 2016.
- [9] N. Le Scouarnec, “Cuckoo++ hash tables: high-performance hash tables for networking applications,” in *Proceedings of the 2018 Symposium on Architectures for Networking and Communications Systems (ANCS)*, pp. 41–54, ACM, 2018.
- [10] B. Minaud and C. Papamanthou, “Generalized cuckoo hashing with a stash, revisited,” *Information Processing Letters*, vol. 181, p. 106356, 2023.
- [11] D. Noble, “Explicit, closed-form, general bounds for cuckoo hashing with a stash.” Cryptology ePrint Archive, Paper 2021/447, 2021. <https://eprint.iacr.org/2021/447>.
- [12] W. Kuszmaul and M. Mitzenmacher, “Efficient d-ary cuckoo hashing at high load factors by bubbling up,” in *Proceedings of the 2025 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 3931–3952, 2025.
- [13] C. Megías, A. Forench, E. Ros, and J. Díaz, “Minimizing cuckoo hashing insertion time for networking applications in fpgas,” *IEEE Access*, vol. 13, pp. 216832–216841, 2025.
- [14] B. Xiong, G. Hu, S. Liu, J. Zhao, J. Zhang, B. Zhao, and K. Li, “TECache: Traffic-aware energy-saving cache with optimal utilization for TCAM flow tables in SDN data plane,” *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4257–4273, 2025.
- [15] R. R. K. Tripathi, P. K. Singh, and S. Singh, “Stable and low associative left-right hashing,” *Computer Science*, vol. 26, no. 2, pp. 151–189, 2025.